

Übungsblatt 3: Einführung FreeRTOS und Präemption

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Übung vollziehen wir den Wechsel vom Bare-Metal-C-Code zu einem professionellen Echtzeitbetriebssystem (FreeRTOS) auf dem STM32H755 (Cortex-M4). Sie konfigurieren die Toolchain, legen Tasks mit unterschiedlichen Prioritäten an und verifizieren die Konzepte der **Präemption** und der **Starvation** (Verhungern von Tasks) live auf der Hardware.

Aufgabe 1: Toolchain-Setup für FreeRTOS

Bevor wir Code schreiben, müssen wir das Betriebssystem und die Zeitbasis in der STM32CubeIDE konfigurieren.

1. Legen Sie ein neues Projekt für das Board **STM32H755ZI-Q** an.
2. Aktivieren Sie unter *Middleware and Software Packs* FreeRTOS für den Cortex-M4. Nutzen Sie das Interface **CMSIS_V2**.
3. Stellen Sie die **TOTAL_HEAP_SIZE** in den FreeRTOS-Einstellungen auf mindestens 16384 Bytes (16 KB), um Speicherprobleme beim Anlegen mehrerer Tasks zu vermeiden.
4. Das CubeMX-Tool wird eine Warnung bezüglich der Timebase ausgeben. Ändern Sie unter *System Core* → *SYS* → *Timebase Source* den Timer von **SysTick** auf **TIM6**. Erklären Sie kurz, warum dieser Schritt für die Stabilität des RTOS zwingend notwendig ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Erklärung des Timebase-Wechsels:

In einem STM32-Projekt nutzt die Hardware-Abstraktionsschicht (HAL) standardmäßig den **SysTick**-Timer für Funktionen wie `HAL_Delay()`. Gleichzeitig benötigt FreeRTOS den SysTick **exklusiv** als Herzschlag für den Scheduler, um die 1 ms-Ticks zu generieren. Greifen beide auf denselben Timer zu, überschreiben sich die Handler bzw. Tick-Konfigurationen — zur Laufzeit kommt es unweigerlich zu Timing-Fehlern oder Abstürzen.

Durch die Umstellung der HAL-Timebase auf **TIM6** nutzt die HAL fortan einen separaten Hardware-Timer; der SysTick steht ausschließlich FreeRTOS zur Verfügung (Scheduler, `osDelay`, Tickless Idle).

Aufgabe 2: Task-Design und Generierung

Legen Sie im Configurator (Reiter *Tasks and Queues*) die folgenden drei Tasks an und generieren Sie anschließend den Code:

1. **HeartbeatTask**: Priorität `osPriorityNormal`
2. **HeavyMathTask**: Priorität `osPriorityBelowNormal`
3. **EmergencyTask**: Priorität `osPriorityHigh` (bleibt in dieser Übung blockiert — siehe Aufgabe 3)

Prüfen Sie nach der Code-Generierung, in welcher Dateistruktur die Tasks in der `main.c` angelegt wurden.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Nach der Generierung finden sich in der `main.c` die CMSIS-Aufrufe `osThreadNew()`, welche intern die TCBs und Stacks für die drei Tasks auf dem FreeRTOS-Heap allozieren. Am Ende der Datei liegen die leeren C-Skelette für die Task-Funktionen (typischerweise `StartHeartbeatTask`, `StartHeavyMathTask`, `StartEmergencyTask` — die genauen Namen hängen von der CubeMX-Benennung ab).

Die Prioritätsreihenfolge für den späteren Präemptions-Beweis lautet:

Emergency (High) > Heartbeat (Normal) > HeavyMath (BelowNormal)

Die `EmergencyTask` ist Platzhalter für spätere Laborteile. Wichtig: Eine leere High-Prio-Endlosschleife würde `Heartbeat` und `HeavyMath` aushungern. Deshalb muss `Emergency` in Aufgabe 3 bewusst *Blocked* bleiben (z.,B. `osDelay(osWaitForever)`). Entscheidend für den Präemptions-Beweis ist die Relation `Heartbeat > HeavyMath`.

Aufgabe 3: Präemption beweisen

Füllen Sie nun die generierten Task-Skelette mit Leben, um das präemptive Verhalten von FreeRTOS zu testen.

1. Konfigurieren Sie eine LED (z. B. Pin **PE1**) als `GPIO_Output` und weisen Sie sie dem Cortex-M4 zu, falls noch nicht geschehen.
2. Lassen Sie in der `HeartbeatTask` die LED alle 500 ms umschalten. Verwenden Sie **zwingend** `osDelay(500)` und **nicht** `HAL_Delay()`.
3. Programmieren Sie in der `HeavyMathTask` eine reine Endlosschleife, die eine Variable hochzählt (z. B. `volatile uint32_t counter = 0; while(1){ counter++; }`). Verwenden Sie hier **kein** Delay!
4. In der `EmergencyTask` rufen Sie lediglich `osDelay(osWaitForever)`; auf (Task bleibt *Blocked* und verbraucht keine CPU). Ohne diesen Schritt würde die High-Prio-Emergency den Präemptions-Beweis zerstören.
5. Flashen Sie den Code und beobachten Sie die LED. Erklären Sie, warum das System nicht einfriert, obwohl die `HeavyMathTask` 100 % CPU-Last fordert und die CPU niemals freiwillig abgibt.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Code-Implementierung (in den vorgesehenen USER CODE-Blöcken):

```
/* USER CODE BEGIN Header_StartHeartbeatTask */
void StartHeartbeatTask(void *argument)
{
    for (;;) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
        osDelay(500); /* Ticks; bei configTICK_RATE_HZ = 1000 → 500 ms */
    }
}
/* USER CODE END Header_StartHeartbeatTask */

/* USER CODE BEGIN Header_StartHeavyMathTask */
void StartHeavyMathTask(void *argument)
{
    volatile uint32_t counter = 0;
    for (;;) {
        counter++; /* 100 % CPU-Last, kein osDelay() */
    }
}
/* USER CODE END Header_StartHeavyMathTask */

/* USER CODE BEGIN Header_StartEmergencyTask */
void StartEmergencyTask(void *argument)
{
    for (;;) {
        osDelay(osWaitForever); /* Blocked: keine CPU-Last */
    }
}
```

```

    }
}
/* USER CODE END Header_StartEmergencyTask */

```

Warum osDelay statt HAL-Busy-Wait?

HAL_Delay() ist eine Busy-Wait-Schleife: die Task bleibt im Zustand *Running* und blockiert die CPU. Dagegen setzt osDelay() die Task in den Zustand *Blocked*; die CPU-Auslastung dieser Task ist dabei praktisch 0%, und andere Ready-Tasks (hier HeavyMath) dürfen rechnen.

Erklärung des Präemptions-Beweises:

Das System friert nicht ein, weil FreeRTOS ein **präemptives** Betriebssystem ist. Die HeartbeatTask hat eine höhere Priorität (osPriorityNormal) als die HeavyMathTask (osPriorityBelowNormal). Nach 500ms weckt der SysTick-Timer die Heartbeat-Task. Der Scheduler erkennt die höhere Priorität und verdrängt (präemptiert) die rechnende HeavyMath-Task über den **PendSV**-Handler von der CPU. Nach dem Toggle und dem anschließenden osDelay() geht Heartbeat wieder nach *Blocked*, und HeavyMath darf im Hintergrund weiterrechnen.

Aufgabe 4: Starvation (Verhungern) durch Prioritäten

Wir sabotieren nun das System, um die Gefahr falsch gesetzter Prioritäten (RMS-Regelverletzung) zu demonstrieren.

1. Öffnen Sie den CubeMX-Configurator und ändern Sie die Priorität der HeavyMathTask auf osPriorityHigh (höher als HeartbeatTask mit osPriorityNormal). Die EmergencyTask bleibt blockiert (osDelay(osWaitForever)).
2. Generieren und flashen Sie den Code erneut.
3. Beobachten Sie die LED und erklären Sie das Phänomen. Handelt es sich hierbei um eine **Priority Inversion**? Begründen Sie Ihre Antwort.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Beobachtung:

Die LED bleibt dunkel und blinkt nicht mehr. Das System wirkt für den Anwender völlig blockiert.

Erklärung (Starvation vs. Priority Inversion):

Dieses Phänomen nennt man **Starvation** (Verhungern). Die HeartbeatTask wacht zwar alle 500ms auf und wechselt in die Ready-Liste, aber der Scheduler entscheidet, dass die unendliche Schleife der HeavyMathTask eine höhere Priorität (**High**) besitzt. Da HeavyMath niemals ein blockierendes Ereignis aufruft (kein osDelay), gibt sie die CPU niemals ab. Heartbeat bekommt schlichtweg keine Rechenzeit zugewiesen.

Dies ist **keine Priority Inversion**. Bei einer Priority Inversion hält eine Task mit *niedriger* Priorität eine Ressource (Mutex/Schloss) besetzt, auf die eine *hochprior*e Task wartet — eine mitelprior e Task kann die niedrigprior e dann „verdrängen“ und die Hochprior e indirekt ausbremsen.

Hier blockiert hingegen eine wirklich höherpriorisierte Task eine niedriger priorisierte Task rein durch Rechenlast: ein logischer Fehler im Systemdesign (falsche Prioritäten), kein Ressourcenkonflikt.